

JENKINS CICD PIPELINE USING DOCKER

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. J. Lovababu, APSSDC

Submitted by

- 1. Kota Navya Sri, 24BQ1A5488, VVIT Guntur, (Batch-1)**
- 2. Mekala Pavan Kalyan, 24BQ1A54A3, VVIT Guntur, (Batch-1)**
- 3. Gullapalli Aparna, 24BQ1A05F8, VVIT Guntur, (Batch-1)**
- 4. Patapati Sai Abhishek, Y24CB042, RVR &JC, Guntur, (Batch-1)**
- 5. Kattamuri Rishitha Indira Lakshmi, 24BQ1A05J2, VVIT Guntur, (Batch-1)**

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at the **Andhra Pradesh State Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience, and I am truly thankful for the support, guidance, and encouragement I have received throughout the internship.

First and foremost, I extend my sincere gratitude to **Mrs. Bethala Sumana** and **Mr. J. Lovababu**, my mentors and supervisors, for their valuable guidance, continuous encouragement, and unwavering support throughout the development of my project, "**Jenkins CI/CD Pipeline Using Docker.**" Their expertise, constructive feedback, and motivation were instrumental in the successful completion of this project.

I would also like to express my sincere thanks to the entire team at the **Andhra Pradesh State Skill Development Corporation (APSSDC)** for providing a collaborative, innovative, and supportive learning environment. The knowledge, resources, and practical exposure gained during this internship have greatly enhanced my technical skills and understanding of DevOps practices.

Finally, I am grateful to my family and friends for their constant encouragement, patience, and support throughout this learning journey. I am honored to have been a part of this internship program and look forward to applying the skills and knowledge I have gained in my future academic and professional endeavors.

Thank you.

Sincerely,

Kota Navya Sri, 24BQ1A5488, VVIT Guntur, kotnavyasri4312@gmail.com

Mekala Pavan Kalyan, 24BQ1A54A3, VVIT Guntur, mekalakalyan178@gmail.com

Gullapalli Aparna, 24BQ1A05F8, VVIT Guntur, aparnagullapalli265@gmail.com

Patapati Sai Abhishek, Y24CB042, RVR &JC Guntur, saiabhishekipatapati@gmail.com

Kattamuri Rishitha Indira Lakshmi, 24BQ1A05J2, VVIT Guntur, Krishitha0007@gmail.com

ABSTRACT

This project presents the implementation of a **Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins and Docker** to automate the software build, deployment, and delivery process. The primary objective is to streamline application deployment by integrating modern DevOps practices, reducing manual effort, minimizing deployment errors, and ensuring faster and more reliable software releases.

The workflow begins when developers push source code changes to a GitHub repository. Jenkins continuously monitors the repository and automatically triggers the pipeline upon detecting new commits. The pipeline retrieves the latest source code, builds the application, and creates a Docker image using a Dockerfile. The Docker image is then used to deploy the application as a container, ensuring a consistent runtime environment across different systems.

The solution utilizes **GitHub** for source code management, **Jenkins** for CI/CD pipeline automation, **Docker** for containerization, and **Apache Tomcat** as the application server for hosting the web application. SSH is used to securely connect Jenkins with the deployment server, enabling automated deployment without manual intervention.

This automated CI/CD pipeline improves software development efficiency by enabling continuous integration, automated testing and deployment, rapid feedback, and consistent application delivery. Containerization with Docker eliminates environment-specific issues, making deployments more reliable, portable, and scalable.

By implementing DevOps principles and automation, this project demonstrates how Jenkins and Docker can be effectively integrated to build a robust CI/CD pipeline that accelerates software delivery, enhances deployment reliability, and provides a strong foundation for future enhancements such as automated testing, monitoring, cloud deployment, and container orchestration using Kubernetes.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION.....	5-6
2.	OBJECTIVES.....	6-7
3.	METHODOLOGY.....	7-9
4.	TECHNOLOGY STACK.....	9-13
5.	SYSTEM ARCHITECTURE.....	13-22
6.	IMPLEMENTATION.....	22-27
7.	PIPELINE WORKFLOW.....	27-30
8.	RESULTS.....	30-32

1. INTRODUCTION

The rapid growth of software applications has increased the demand for faster development, reliable deployments, and continuous delivery of high-quality software. In traditional software development, application deployment is often performed manually, which can be time-consuming, inconsistent, and prone to human errors. These challenges can delay software releases, increase maintenance efforts, and reduce overall productivity. To address these issues, organizations have adopted **DevOps**, a modern software development approach that emphasizes collaboration, automation, and continuous delivery.

A key practice of DevOps is the implementation of **Continuous Integration (CI)** and **Continuous Deployment (CD)**. Continuous Integration enables developers to frequently integrate code changes into a shared repository, where automated build processes are triggered to verify the application's integrity. Continuous Deployment extends this process by automatically deploying the successfully built application to the target environment. Together, CI/CD pipelines help organizations deliver software more quickly, consistently, and with greater confidence.

This project, "**Jenkins CI/CD Pipeline Using Docker**," demonstrates the practical implementation of an automated software delivery pipeline using industry-standard DevOps tools. The project integrates **GitHub**, **Jenkins**, **Docker**, and **Apache Tomcat** to create a complete automation workflow that minimizes manual intervention. Whenever developers push code changes to a GitHub repository, Jenkins automatically detects the changes, retrieves the latest source code, builds the application, creates a Docker image, and deploys the application inside a Docker container.

Docker plays a crucial role in this project by packaging the application along with all its required dependencies into a lightweight container. This ensures that the application behaves consistently across development, testing, and production environments, eliminating common issues such as dependency conflicts and environment mismatches. Jenkins acts as the automation server that orchestrates the entire workflow, enabling seamless integration between source code management, application building, containerization, and deployment.

The implementation of this CI/CD pipeline significantly reduces deployment time, improves software reliability, and enhances development efficiency by automating repetitive tasks. Developers receive rapid feedback on code changes, while operations teams benefit from

standardized and reproducible deployments. The automated pipeline also reduces the possibility of human error and supports frequent software releases without compromising application quality.

This project provides practical exposure to modern DevOps practices and demonstrates how automation and containerization can transform the software development lifecycle. It serves as a strong foundation for understanding enterprise-level deployment strategies and can be further enhanced by integrating automated testing, monitoring tools, cloud platforms such as AWS, Azure, or Google Cloud, and container orchestration technologies like Kubernetes. Overall, the project highlights the importance of CI/CD pipelines in delivering scalable, reliable, and efficient software solutions in today's fast-paced development environment.

2. OBJECTIVES

The primary objective of this project is to design and implement an automated **Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins and Docker** for a Java-based web application. The project aims to demonstrate how DevOps practices can improve software development by automating the build, testing, and deployment processes, resulting in faster, more reliable, and consistent application delivery.

The specific objectives of this project are as follows:

1. To understand and implement the fundamental concepts of **DevOps, Continuous Integration (CI), and Continuous Deployment (CD)**.
2. To configure **Jenkins** as an automation server for creating and managing CI/CD pipelines.
3. To integrate **GitHub** with Jenkins so that code changes automatically trigger the pipeline.
4. To automate the process of fetching source code, building the application, and deploying it without manual intervention.
5. To create a **Dockerfile** and build Docker images for the application.
6. To containerize the web application using **Docker**, ensuring consistent execution across different environments.
7. To deploy the application inside a Docker container running on an **Apache Tomcat** server.
8. To reduce deployment time, eliminate repetitive manual tasks, and minimize human errors through automation.
9. To improve the reliability, consistency, and efficiency of the software development and deployment lifecycle.

10. To gain practical experience with industry-standard DevOps tools, including **GitHub, Jenkins, Docker, Apache Tomcat, and SSH.**
11. To demonstrate a real-world CI/CD workflow that can be easily extended with automated testing, cloud deployment, monitoring, and container orchestration technologies such as Kubernetes.

By achieving these objectives, the project demonstrates the practical application of DevOps principles in automating software delivery, improving development productivity, and enabling faster, scalable, and reliable application deployments.

3. METHODOLOGY

The methodology followed in this project is based on the **DevOps Continuous Integration and Continuous Deployment (CI/CD)** approach. The project automates the software development and deployment lifecycle using **GitHub, Jenkins, Docker, and Apache Tomcat.** Each stage of the pipeline is designed to reduce manual effort, improve deployment reliability, and ensure that application updates are delivered efficiently.

3.1 Project Planning

The first step involved identifying the project requirements and selecting the appropriate DevOps tools for automation. Jenkins was chosen as the automation server, GitHub as the version control system, Docker for containerization, and Apache Tomcat for hosting the Java web application.

3.2 Source Code Management

The application source code and the Dockerfile were stored in a GitHub repository. Developers make changes to the application and push the updated code to GitHub. This repository serves as the single source of truth for the project and enables version control and collaboration.

3.3 Jenkins Configuration

Jenkins was installed and configured on a Linux server. The required plugins, such as Git and Docker integration plugins, were installed to enable communication with GitHub and Docker. SSH was configured to allow secure access between Jenkins and the deployment environment.

3.4 Continuous Integration

A Jenkins pipeline was created to automate the Continuous Integration process. Whenever new code is pushed to the GitHub repository, Jenkins automatically detects the changes and triggers the pipeline. The pipeline performs the following tasks:

- Retrieves the latest source code from GitHub.
- Builds the Java web application.
- Verifies that the build process completes successfully.
- Prepares the application for deployment.

This automated integration process ensures that code changes are continuously validated and reduces the risk of integration issues.

3.5 Docker Image Creation

After the application is successfully built, Jenkins uses the Dockerfile to create a Docker image containing the web application and all its required dependencies. Docker packages the application into a lightweight container, ensuring that it behaves consistently across different environments.

3.6 Automated Deployment

Once the Docker image is successfully created, Jenkins automatically deploys the application by running a Docker container. The container exposes the application through Apache Tomcat, making it accessible to users through a web browser. Existing containers can be stopped and replaced with the updated version to ensure seamless deployment.

3.7 Testing and Verification

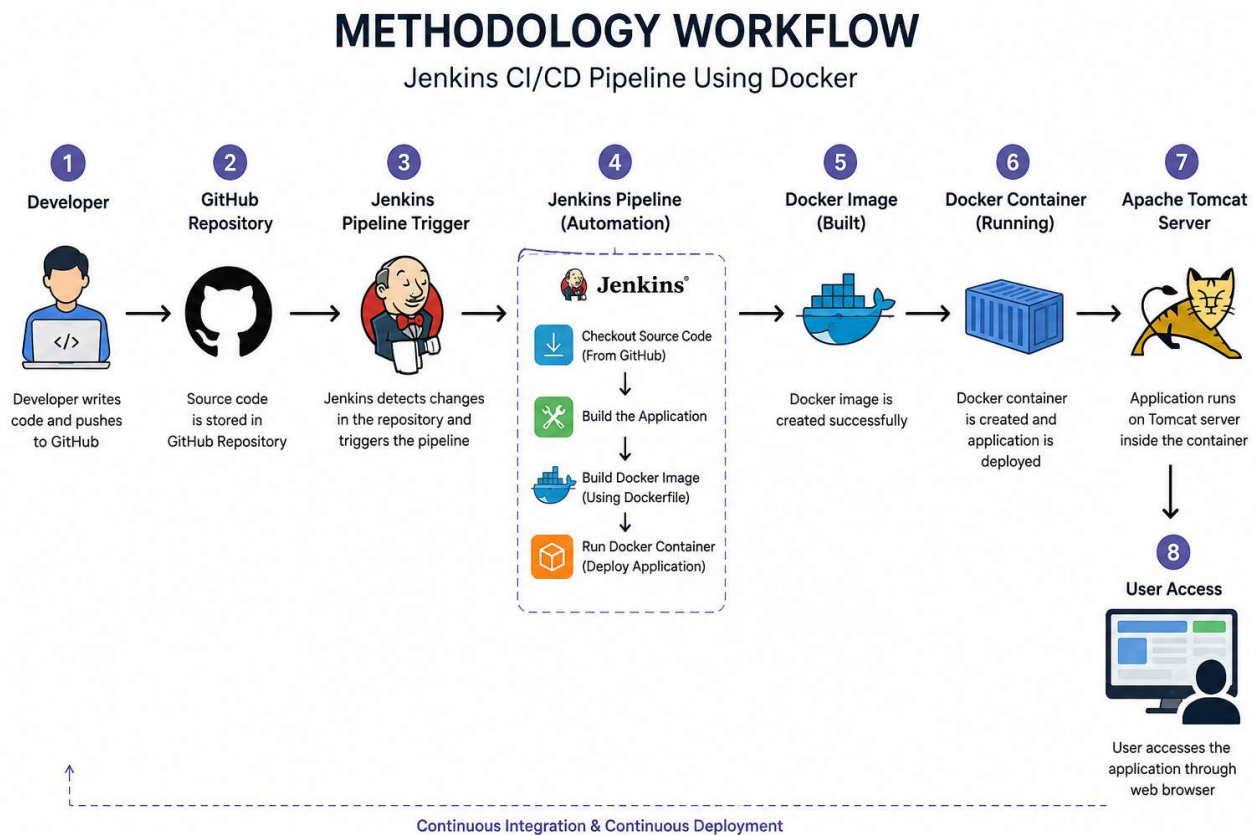
After deployment, the application is tested by accessing it through the configured server port. The successful execution of the application confirms that the CI/CD pipeline has completed all stages correctly. Jenkins console logs and Docker container status are also verified to ensure successful deployment.

3.8 Result Evaluation

The performance of the implemented CI/CD pipeline is evaluated based on successful automation of the build and deployment process. The project demonstrates reduced deployment time, fewer manual errors, improved consistency, and faster software delivery compared to traditional manual deployment methods.

Methodology Workflow

The complete methodology of the project follows the sequence below:



This methodology successfully demonstrates the implementation of a modern DevOps workflow using Jenkins and Docker. By automating software integration and deployment, the project improves efficiency, enhances reliability, and provides a scalable solution for continuous software delivery.

4. TECHNOLOGY STACK

The successful implementation of the **Jenkins CI/CD Pipeline Using Docker** project relies on a set of modern DevOps tools and technologies. Each technology plays a specific role in automating the software development, integration, containerization, and deployment process. The technology stack used in this project is described below.

4.1 GitHub

GitHub is a cloud-based version control platform used to store and manage the application's source code. It enables developers to track code changes, collaborate efficiently, and maintain different versions of the project. In this project, GitHub serves as the central repository where the application code is stored. Whenever new code is pushed to the repository, Jenkins automatically detects the changes and triggers the CI/CD pipeline.

Features

- Source code management
- Version control using Git
- Collaboration and code sharing
- Integration with Jenkins through webhooks or polling

4.2 Jenkins

Jenkins is an open-source automation server used to implement Continuous Integration and Continuous Deployment (CI/CD). It automates the software build and deployment process, eliminating manual intervention. Jenkins continuously monitors the GitHub repository and executes the pipeline whenever changes are detected.

Features

- Continuous Integration and Continuous Deployment
- Automated build execution
- Pipeline creation using Jenkinsfile
- Plugin support for Git, Docker, and SSH
- Build monitoring and reporting

4.3 Docker

Docker is a containerization platform that packages the application and its dependencies into lightweight containers. Containers ensure that the application runs consistently across development, testing, and production environments, eliminating environment-related issues.

Features

- Application containerization
- Portable deployment

- Fast startup and execution
- Lightweight compared to virtual machines
- Consistent runtime environment

4.4 Apache Tomcat

Apache Tomcat is an open-source Java Servlet container and web server used to deploy Java web applications. In this project, the application is deployed inside a Docker container running Apache Tomcat.

Features

- Java web application hosting
- Servlet and JSP support
- Lightweight application server
- Easy deployment of WAR files

4.5 Git

Git is a distributed version control system used to manage the application's source code locally before pushing changes to GitHub. It helps developers maintain project history and collaborate effectively.

Features

- Source code version control
- Branching and merging
- Commit history tracking
- Fast and efficient collaboration

4.6 Dockerfile

A **Dockerfile** is a text file that contains instructions for building a Docker image. It specifies the base image, application files, dependencies, and startup commands required to create a runnable Docker container.

Features

- Automates Docker image creation
- Defines container configuration
- Ensures reproducible builds
- Simplifies deployment

4.7 Linux (Ubuntu)

The project environment is hosted on an **Ubuntu Linux** server. Jenkins, Docker, Git, and Apache Tomcat are installed and configured on this operating system.

Features

- Stable and secure operating system
- Efficient resource utilization
- Excellent support for DevOps tools
- Command-line administration

4.8 SSH (Secure Shell)

SSH is used to establish a secure connection between Jenkins and the deployment server. It enables Jenkins to execute deployment commands remotely without exposing sensitive information.

Features

- Secure remote access
- Encrypted communication
- Remote command execution
- Authentication using SSH keys

4.9 Java

The web application is developed using **Java**, one of the most widely used programming languages for enterprise applications. The application is packaged as a WAR file and deployed on Apache Tomcat.

Features

- Platform-independent

- Object-oriented programming
- High performance
- Enterprise application development

Technology Stack Summary

TECHNOLOGY STACK

S.No.	Technology	Logo	Purpose	Key Features
1.	GitHub		Source code repository and version control	• Stores and manages code • Enables collaboration • Webhooks integration with Jenkins
2.	Git		Distributed version control system	• Track changes in source code • Branching and merging • Maintains code history
3.	Jenkins		Automation server for CI/CD pipeline	• Automates build, test and deploy • Pipeline creation using Jenkinsfile • Extensive plugin support
4.	Docker		Containerization platform for applications	• Packages application and dependencies • Lightweight and portable containers • Consistent environment
5.	Dockerfile		Configuration file to build Docker images	• Defines base image and dependencies • Instructions to build image • Reproducible builds
6.	Apache Tomcat		Web server / Servlet container for Java apps	• Hosts Java web applications • Supports Servlet and JSP • Deploys WAR files
7.	Java		Programming language used for application development	• Platform independent • Object-oriented • High performance
8.	Ubuntu Linux		Operating system for hosting and deployment	• Stable and secure • Supports DevOps tools • Efficient resource management
9.	SSH (Secure Shell)		Secure connection between Jenkins and server	• Secure remote access • Encrypted communication • Key-based authentication

The combination of these technologies enables the successful implementation of an automated CI/CD pipeline. Together, they streamline the software development lifecycle by automating code integration, containerization, and deployment, resulting in faster, more reliable, and consistent software delivery.

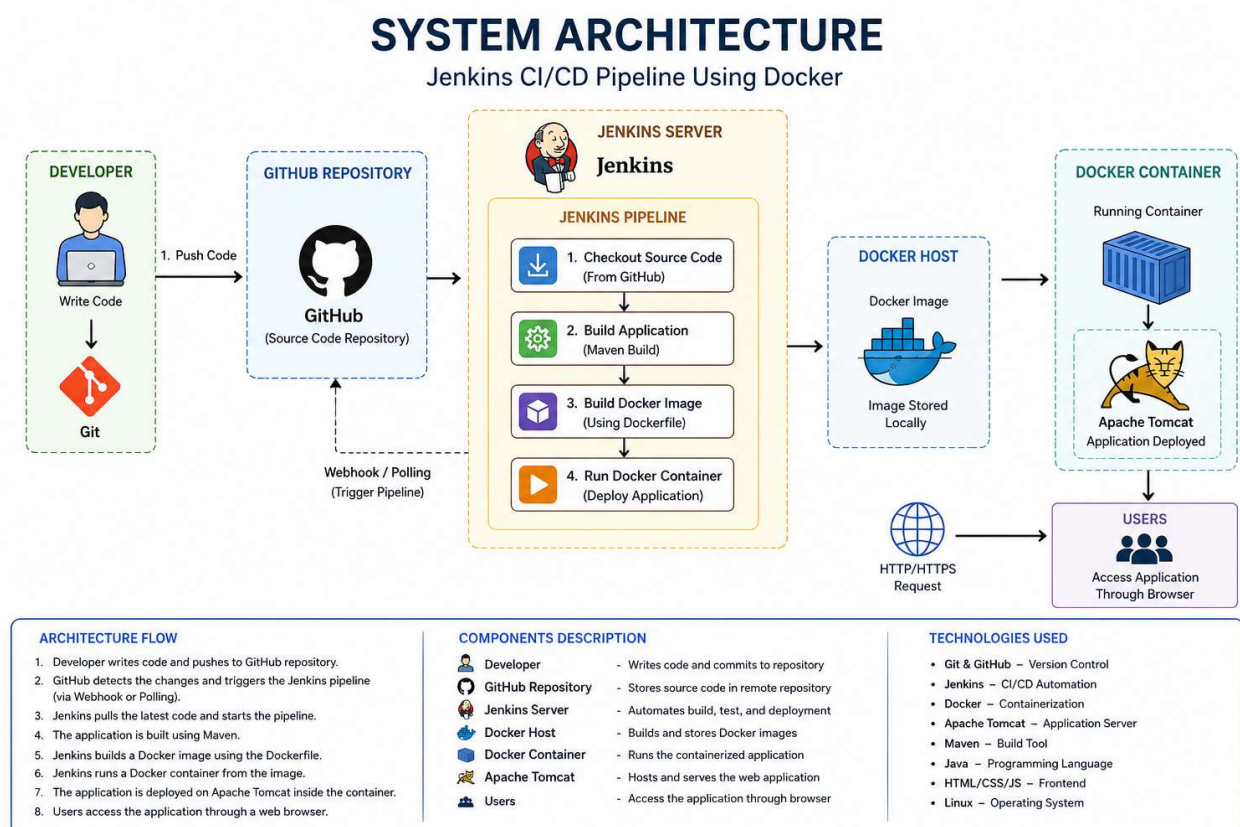
5.SYSTEM ARCHITECTURE

The **Jenkins CI/CD Pipeline Using Docker** project is designed as an automated DevOps solution that streamlines the software build and deployment process. The architecture integrates **GitHub**, **Jenkins**, **Docker**, and **Apache Tomcat** to enable Continuous Integration and

Continuous Deployment (CI/CD). It automates the workflow from source code management to application deployment, reducing manual effort, minimizing deployment errors, and ensuring faster software delivery.

The system follows a structured architecture consisting of **source code management, build automation, containerization, and deployment layers**. Developers push application code to a GitHub repository, where Jenkins continuously monitors for changes. When new code is detected, Jenkins automatically builds the application, creates a Docker image, and deploys it as a Docker container running on Apache Tomcat. Users can then access the deployed application through a web browser.

This automated architecture provides a consistent deployment environment, improves software quality, enhances development productivity, and serves as a scalable foundation for modern DevOps practices.



5.1 Source Code Management – GitHub

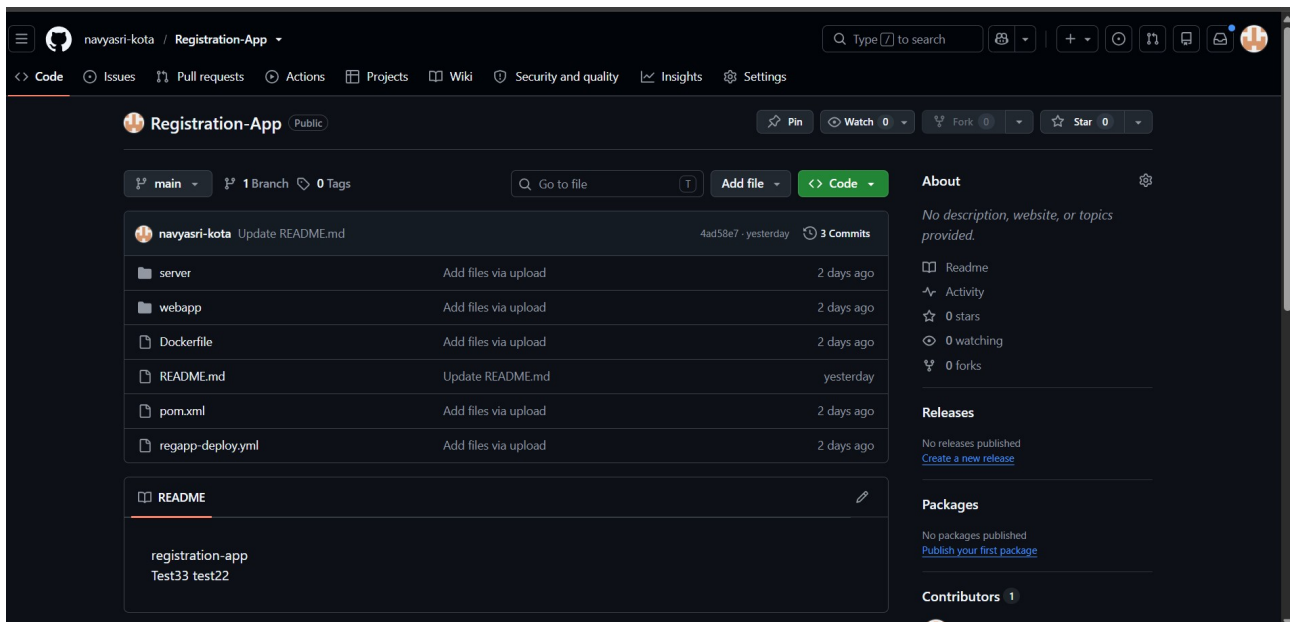
The development process begins with the developer writing or modifying the application source code. The updated code is committed and pushed to a GitHub repository, which acts as the central

source code management platform. Jenkins continuously monitors this repository using Poll SCM or GitHub Webhooks. Whenever new code is detected, the CI/CD pipeline is automatically triggered.

Key Features:

- Centralized source code repository
- Git version control
- Automatic pipeline triggering
- Collaboration among developers

Figure 1. Source Code Files



5.2 Continuous Integration – Jenkins

Jenkins serves as the automation server that executes the CI/CD pipeline. After detecting changes in GitHub, Jenkins checks out the latest source code and starts the build process. The pipeline is configured using Jenkins jobs or a Jenkinsfile to automate each stage of the software delivery lifecycle.

Key Features:

- Automatic build triggering
- Pipeline automation

- Integration with GitHub
- Build history and console logs

Figure 2. Jenkins Dashboard

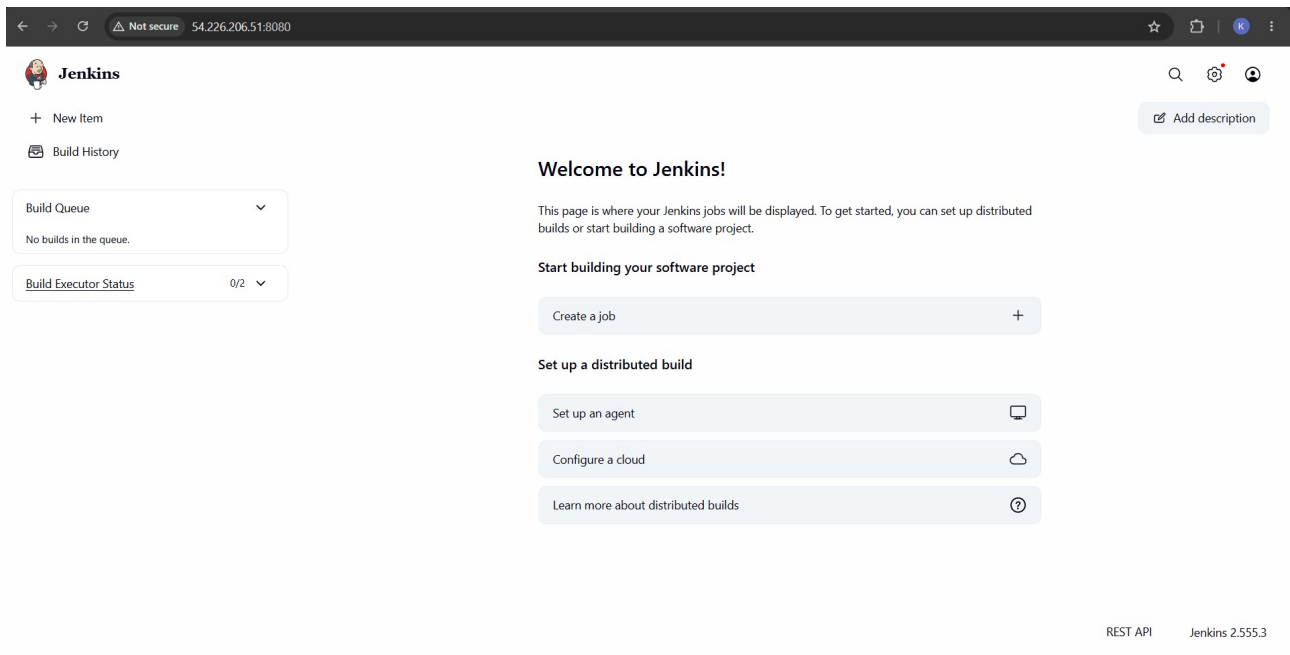


Figure 3. Jenkins CI/CD Pipeline Job

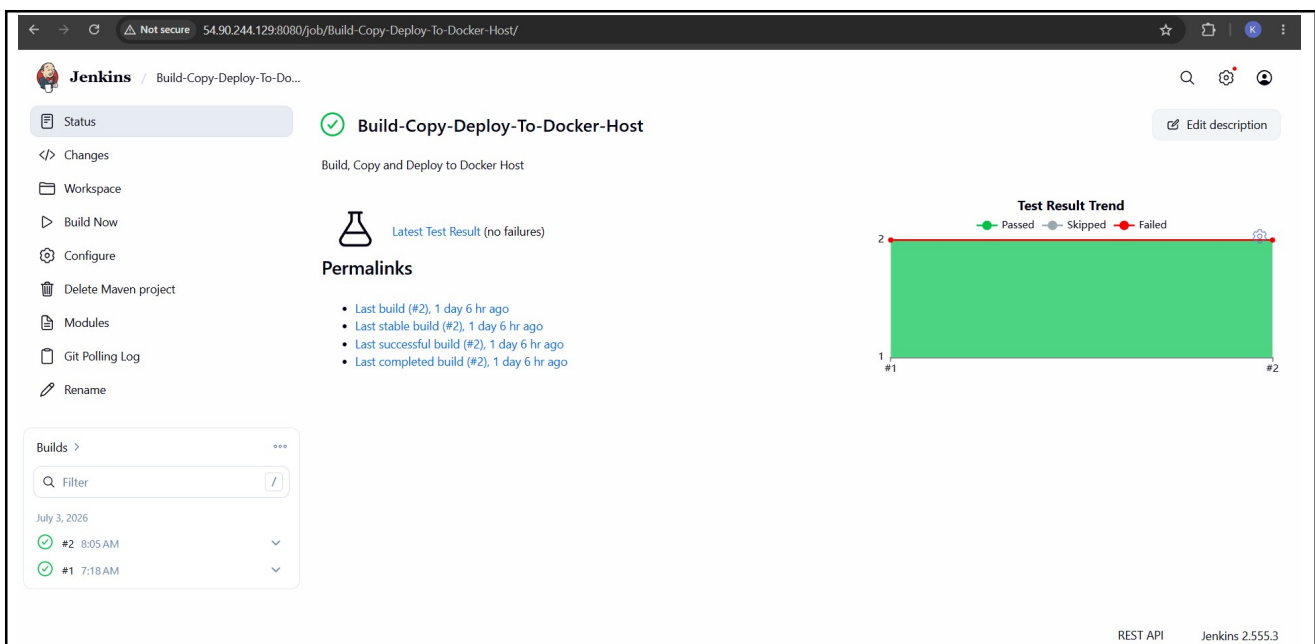


Figure 4. Build History

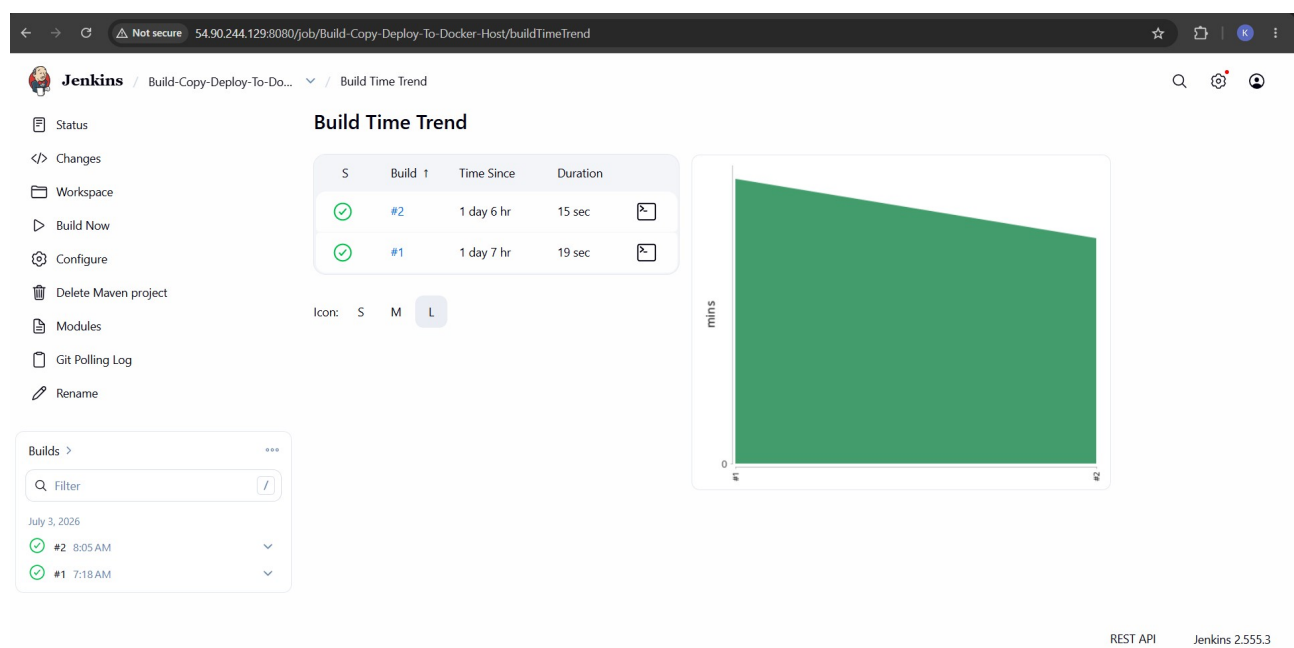
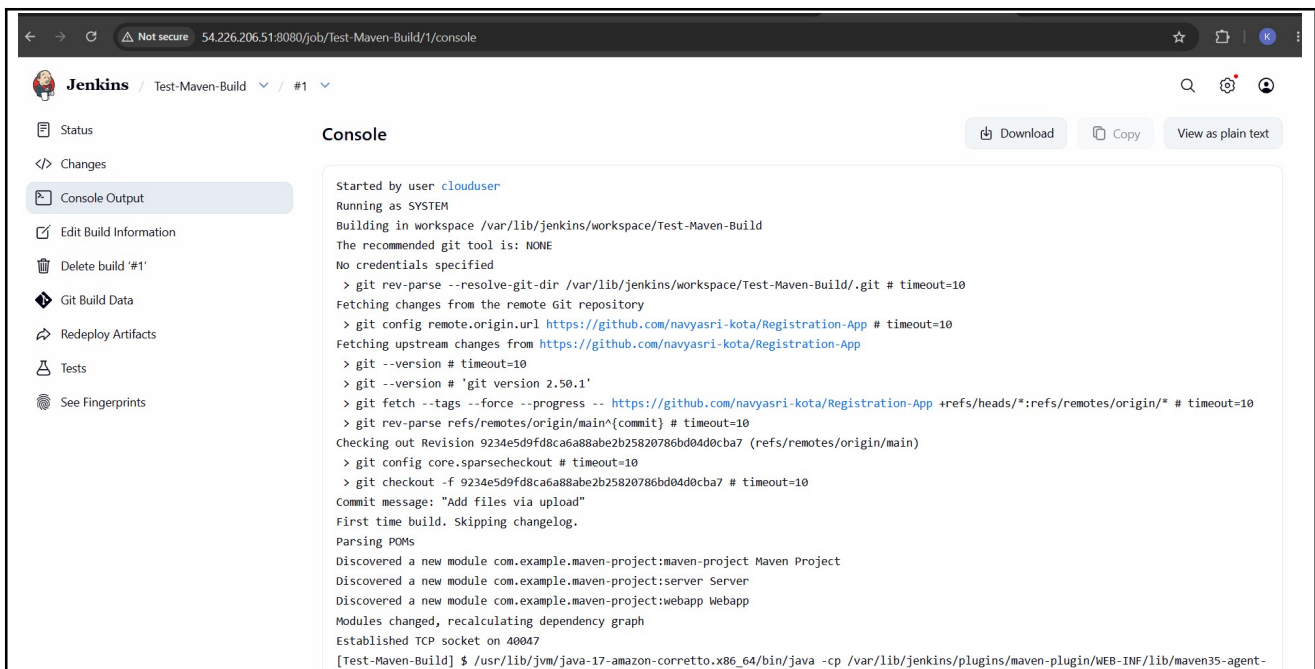


Figure 5. Console Output

The image shows a Jenkins web interface. On the left is a sidebar with navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#1', Git Build Data, Redeploy Artifacts, Tests, and See Fingerprints. The main area is titled 'Console' and displays the output of a build. The output shows the build starting as 'clouduser' on 'SYSTEM', building in the workspace '/var/lib/jenkins/workspace/Test-Maven-Build'. It then performs a git checkout of a specific revision, configures the Maven project, and lists discovered modules like 'com.example.maven-project:maven-project', 'com.example.maven-project:server', and 'com.example.maven-project:webapp'. The build ends with the command to run the application using Java.

```
Started by user clouduser
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Test-Maven-Build
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Test-Maven-Build/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/navyasri-kota/Registration-App # timeout=10
Fetching upstream changes from https://github.com/navyasri-kota/Registration-App
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/navyasri-kota/Registration-App +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9234e5d9fd8ca6a88abe2b25820786bd04d0cba7 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9234e5d9fd8ca6a88abe2b25820786bd04d0cba7 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
Parsing POMs
Discovered a new module com.example.maven-project:maven-project Maven Project
Discovered a new module com.example.maven-project:server Server
Discovered a new module com.example.maven-project:webapp Webapp
Modules changed, recalculating dependency graph
Established TCP socket on 40047
[Test-Maven-Build] $ /usr/lib/jvm/java-17-amazon-corretto.x86_64/bin/java -cp /var/lib/jenkins/plugins/maven-plugin/WEB-INF/lib/maven35-agent-
```

5.3 Containerization – Docker

After the application is successfully built, Jenkins executes the Dockerfile to create a Docker image. Docker packages the application and its dependencies into a lightweight container, ensuring consistent execution across different environments.

Key Features:

- Docker image creation
- Portable deployment
- Lightweight containers
- Environment consistency
-

Figure 6. Dockerfile

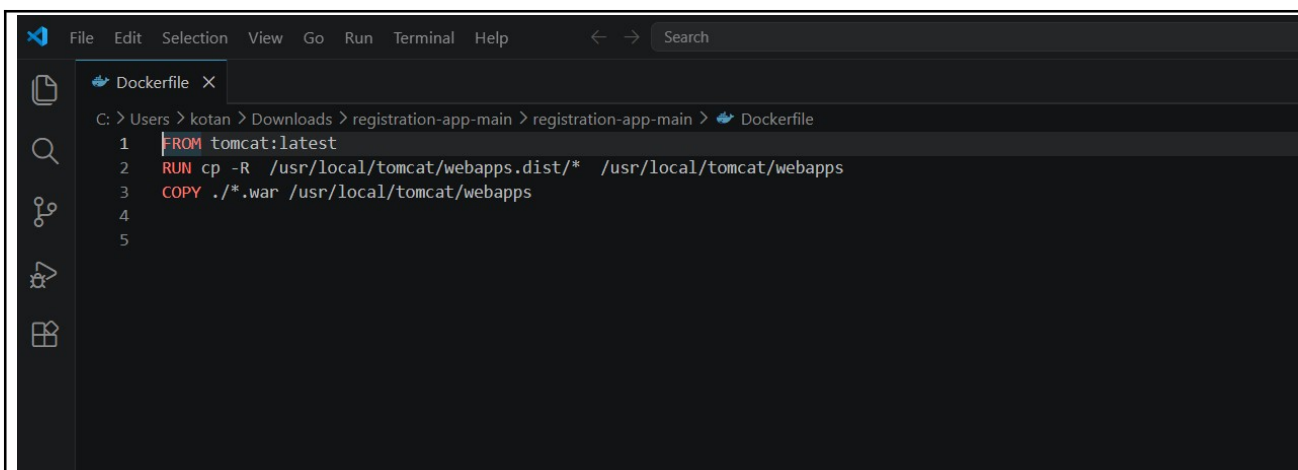


Figure 7. Docker Images

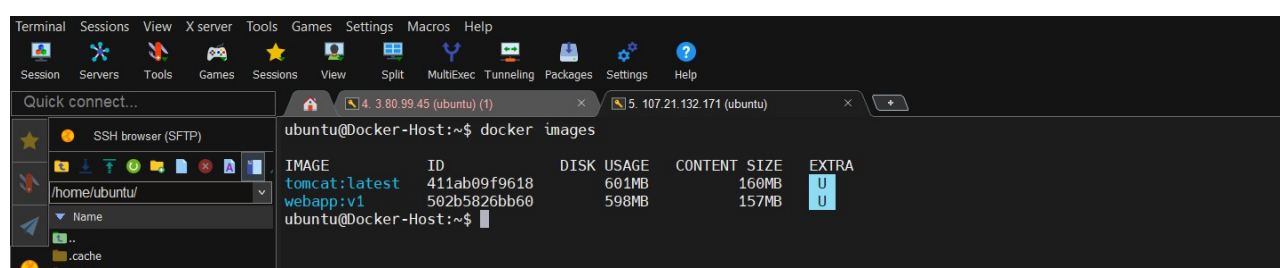
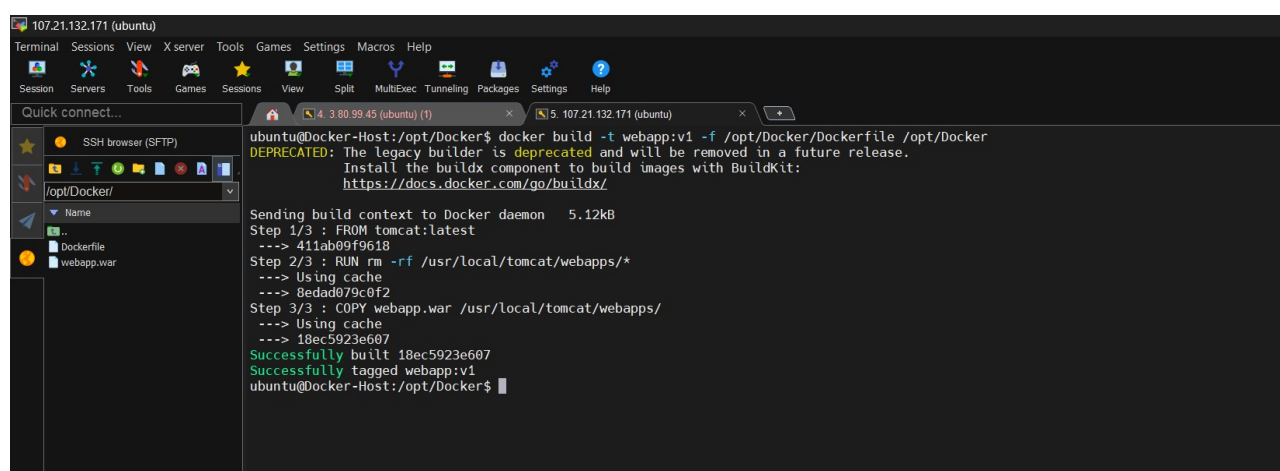


Figure 9. Docker Build Process



5.4 Deployment – Docker Container and Apache Tomcat

Jenkins automatically deploys the application by creating a Docker container from the generated Docker image. Apache Tomcat inside the container hosts the Java web application, making it accessible through a web browser.

Key Features:

- Automated deployment
- Container-based hosting
- Apache Tomcat server
- Fast application startup
-

Figure 9. Running Docker Containers

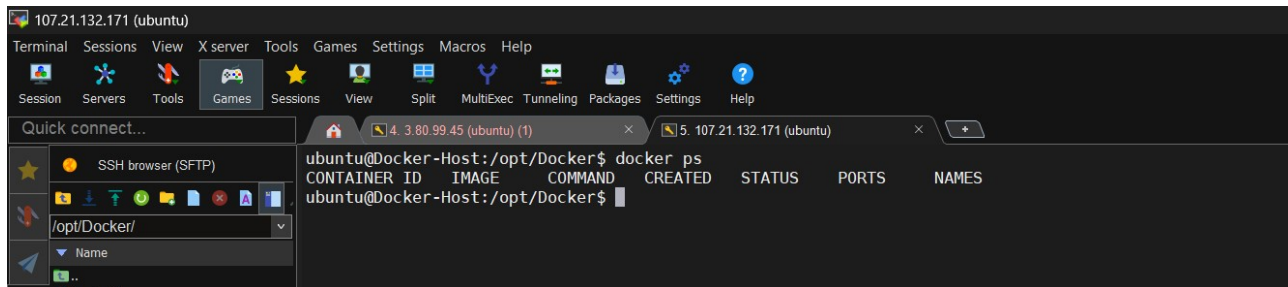


Figure 10. Apache Tomcat Running

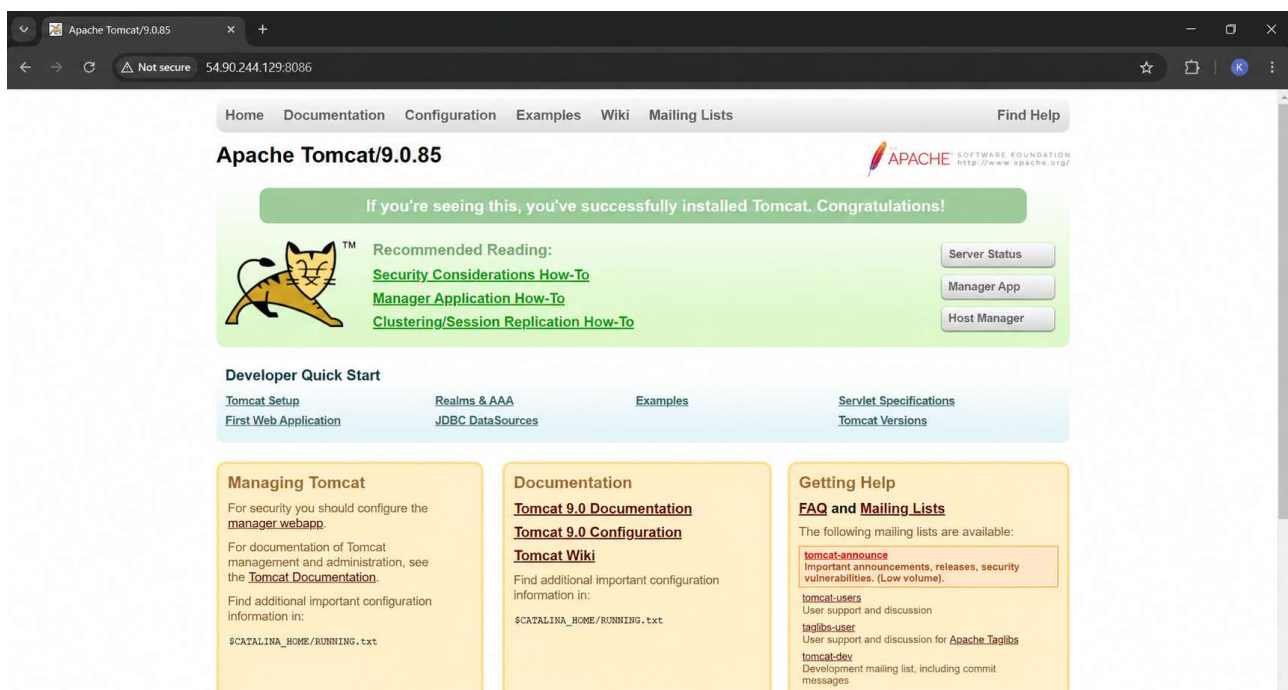
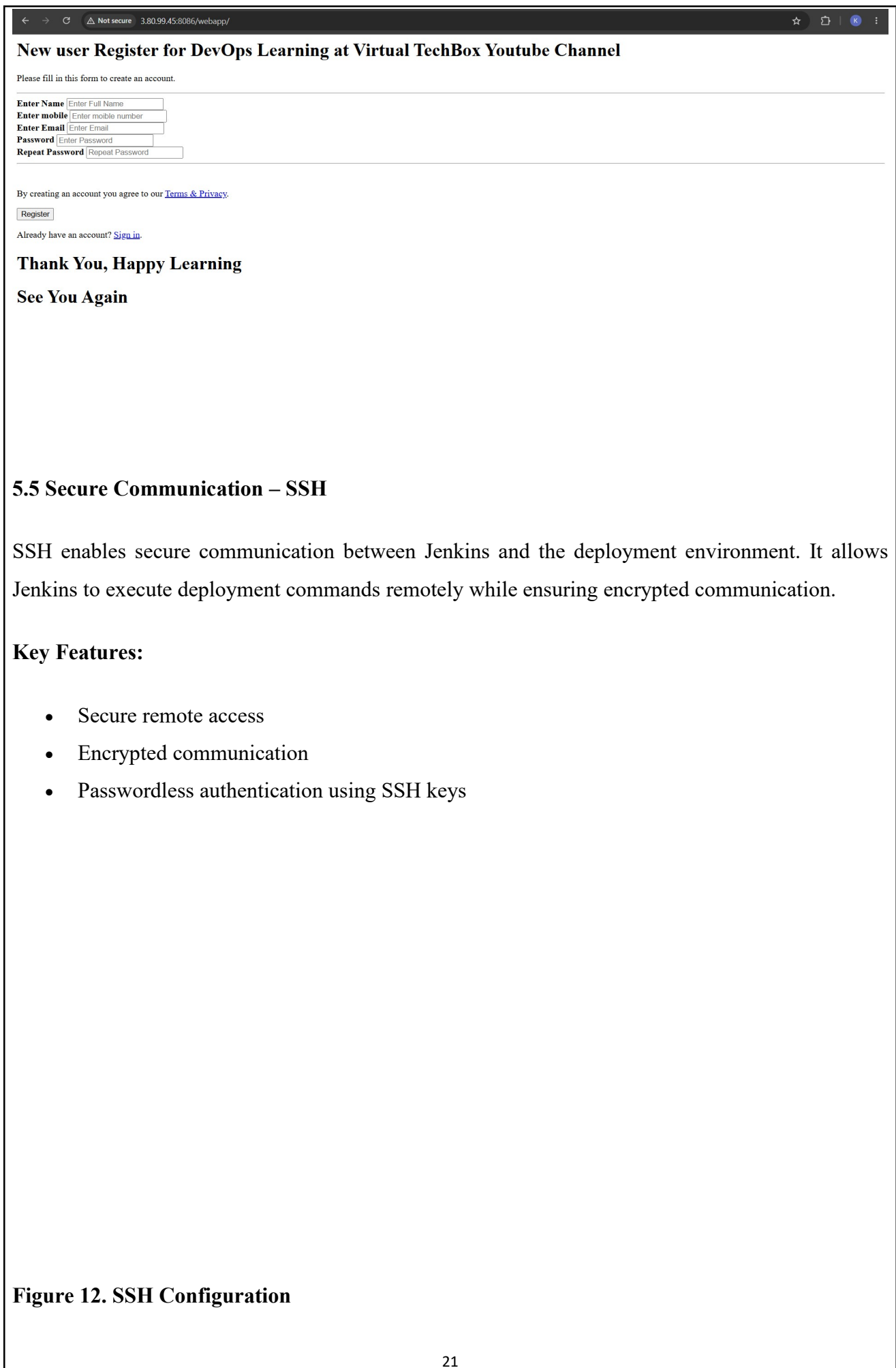


Figure 11. Application Successfully Deployed



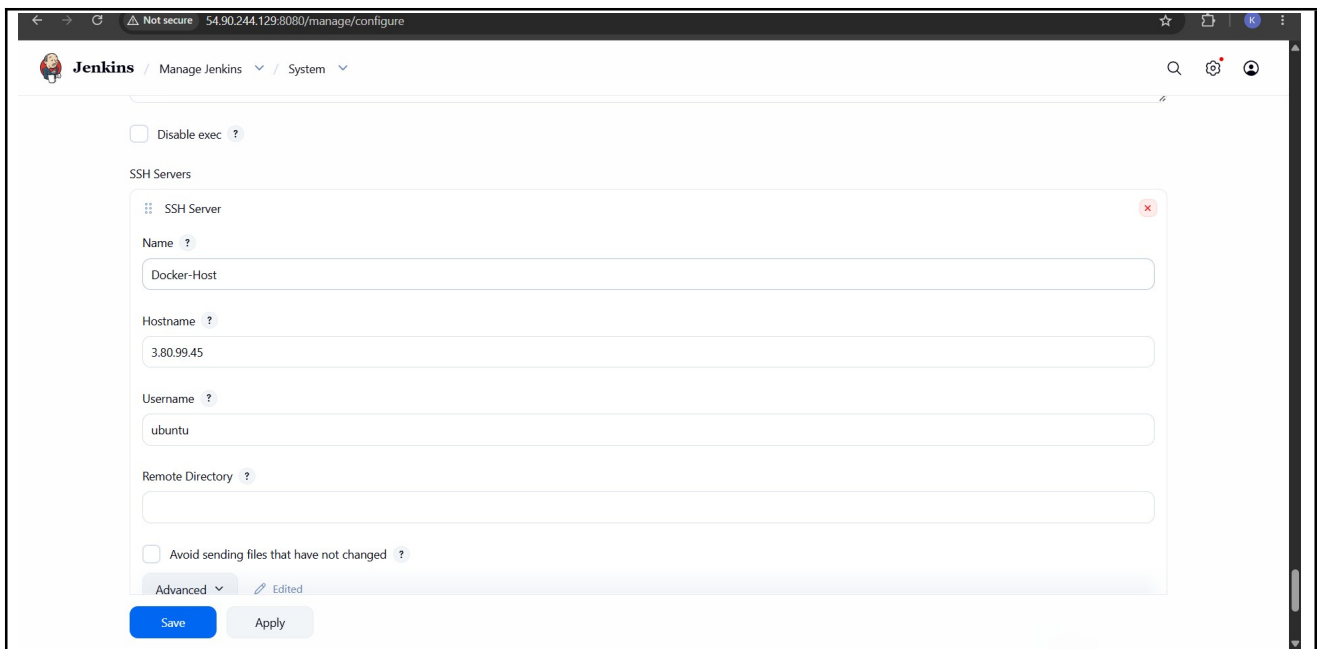
5.5 Secure Communication – SSH

SSH enables secure communication between Jenkins and the deployment environment. It allows Jenkins to execute deployment commands remotely while ensuring encrypted communication.

Key Features:

- Secure remote access
- Encrypted communication
- Passwordless authentication using SSH keys

Figure 12. SSH Configuration



6.IMPLEMENTATION

The implementation of the **Jenkins CI/CD Pipeline Using Docker** project focuses on automating the software build and deployment process using DevOps tools. The project integrates **GitHub, Jenkins, Docker, and Apache Tomcat** to create a seamless Continuous Integration and Continuous Deployment (CI/CD) workflow. The implementation is divided into several phases, each performing a specific task in the automation process.

6.1 Project Setup

The implementation began with the creation of an Ubuntu-based Amazon EC2 instance. The required software packages, including Java Development Kit (JDK), Git, Jenkins, and Docker, were installed and configured. After installation, the Jenkins and Docker services were started, and the Jenkins user was added to the Docker group to enable Docker command execution from Jenkins.

Commands Used:

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk -y
```

```
sudo apt install git -y
```

```
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
sudo systemctl start jenkins
```

```
sudo systemctl enable jenkins
```

```
sudo usermod -aG docker jenkins
```

```
sudo systemctl restart jenkins
```

6.2 GitHub Repository Configuration

The application source code along with the Dockerfile was uploaded to a GitHub repository. GitHub acts as the centralized version control platform, enabling developers to track changes and collaborate effectively. Jenkins was configured to access this repository and automatically trigger the CI/CD pipeline whenever new code was pushed.

Git Commands Used:

```
git init
```

```
git add .
```

```
git commit -m "Initial Commit"
```

```
git branch -M main
```

```
git remote add origin https://github.com/username/repository.git
```

```
git push -u origin main
```

6.3 Jenkins Job Configuration

A Jenkins Freestyle (or Pipeline) job was created to automate the software build and deployment process. The GitHub repository URL was configured in the Source Code Management section, while **Poll SCM** was enabled to monitor the repository for changes. An **Execute Shell** build step was added to execute the required build and deployment commands automatically.

Build Commands:

```
mvn clean package
```

```
docker build -t webapp:v1 .
```

```
docker stop Registration-App || true
```

```
docker rm Registration-App || true
```

```
docker run -d --name Registration-App -p 8086:8080 webapp:v1
```

These commands build the application, create a Docker image, stop any existing container, remove it, and deploy the updated application automatically.

6.4 Docker Image Creation

After the application was successfully built, Jenkins executed the Dockerfile to create a Docker image. Docker packages the application together with Apache Tomcat and all required dependencies, ensuring consistent execution across different environments.

Command Used to Build Docker Image:

```
docker build -t webapp:v1 .
```

Command Used to Verify Docker Images:

```
docker images
```

6.5 Automated Application Deployment

Once the Docker image was created, Jenkins deployed the application by creating a Docker container. Before launching the updated container, any previously running container was stopped and removed to avoid conflicts.

Deployment Commands:

```
docker stop Registration-App
```

```
docker rm Registration-App
```

```
docker run -d --name Registration-App -p 8086:8080 webapp:v1
```

Command to Verify Running Containers:

```
docker ps
```

6.6 Application Verification

After deployment, the application was verified by accessing it through a web browser using the Amazon EC2 public IP address and the mapped port number.

Application URL:

```
http://<EC2-Public-IP>:8086/Registration-App
```

Additional commands were executed to verify the deployment status.

```
docker ps
```

```
docker logs Registration-App
```

```
systemctl status jenkins
```

```
systemctl status docker
```

The successful display of the application confirmed that the CI/CD pipeline had completed all stages successfully.

6.7 Pipeline Execution Flow

The complete implementation of the project follows the workflow shown below.

1. The developer modifies the application source code.
2. The updated code is committed and pushed to the GitHub repository.

```
git add .
```

```
git commit -m "Updated Application"
```

```
git push origin main
```

1. Jenkins detects changes in the GitHub repository through Poll SCM.
2. Jenkins retrieves the latest source code.
3. Jenkins builds the application.

```
mvn clean package
```

1. Docker creates a new image using the Dockerfile.

```
docker build -t webapp:v1 .
```

1. Jenkins stops and removes the existing Docker container.

```
docker stop Registration-App
```

```
docker rm Registration-App
```

1. Jenkins launches a new Docker container.

```
docker run -d --name Registration-App -p 8086:8080 webapp:v1
```

1. Apache Tomcat hosts the application inside the Docker container.
2. Users access the deployed application through a web browser.

```
http://<EC2-Public-IP>:8086/Registration-App
```

The implementation successfully demonstrates the practical application of DevOps by integrating **GitHub, Jenkins, Docker, and Apache Tomcat** into an automated CI/CD pipeline. Automation significantly reduces manual effort, minimizes deployment errors, improves deployment consistency, and accelerates the software delivery process. This implementation provides a reliable and scalable solution for continuous software integration and deployment.

7. PIPELINE WORKFLOW

The CI/CD pipeline is the core component of this project, automating the software development and deployment lifecycle. The pipeline begins when the developer pushes the latest source code to the GitHub repository. Jenkins continuously monitors the repository using **Poll SCM** and automatically triggers the pipeline whenever changes are detected.

Once the pipeline starts, Jenkins checks out the latest source code from GitHub and initiates the build process. After a successful build, Jenkins executes the Dockerfile to create a new Docker image containing the updated application and its required dependencies. If an older Docker container is running, Jenkins stops and removes it before deploying a new container using the latest Docker image. The application is then hosted on **Apache Tomcat** inside the Docker container and becomes accessible to users through a web browser.

This automated workflow eliminates manual deployment tasks, reduces configuration errors, ensures consistent deployments, and enables faster software delivery.

7.1 Pipeline Stages

Stage 1: Source Code Development

The developer writes or modifies the application source code and commits the changes using Git.

Command:

```
git add .
```

```
git commit -m "Updated Application"
```

```
git push origin main
```

Stage 2: Source Code Management

The updated source code is pushed to the GitHub repository. GitHub stores the latest version of the application and serves as the source repository for Jenkins.

Stage 3: Jenkins Pipeline Trigger

Jenkins continuously monitors the GitHub repository using **Poll SCM**. When new code is detected, Jenkins automatically triggers the CI/CD pipeline.

Stage 4: Build Process

Jenkins retrieves the latest source code and builds the Java web application using Maven.

Command:

```
mvn clean package
```

The generated WAR file is prepared for deployment.

Stage 5: Docker Image Creation

After the build is completed successfully, Jenkins executes the Dockerfile to build a Docker image.

Command:

```
docker build -t webapp:v1 .
```

The Docker image packages the application together with Apache Tomcat and all required dependencies.

Stage 6: Container Deployment

The existing Docker container is stopped and removed before deploying the updated application.

Commands:

```
docker stop Registration-App
```

```
docker rm Registration-App
```

```
docker run -d --name Registration-App -p 8086:8080 webapp:v1
```

Stage 7: Application Verification

After deployment, the running container is verified.

Command:

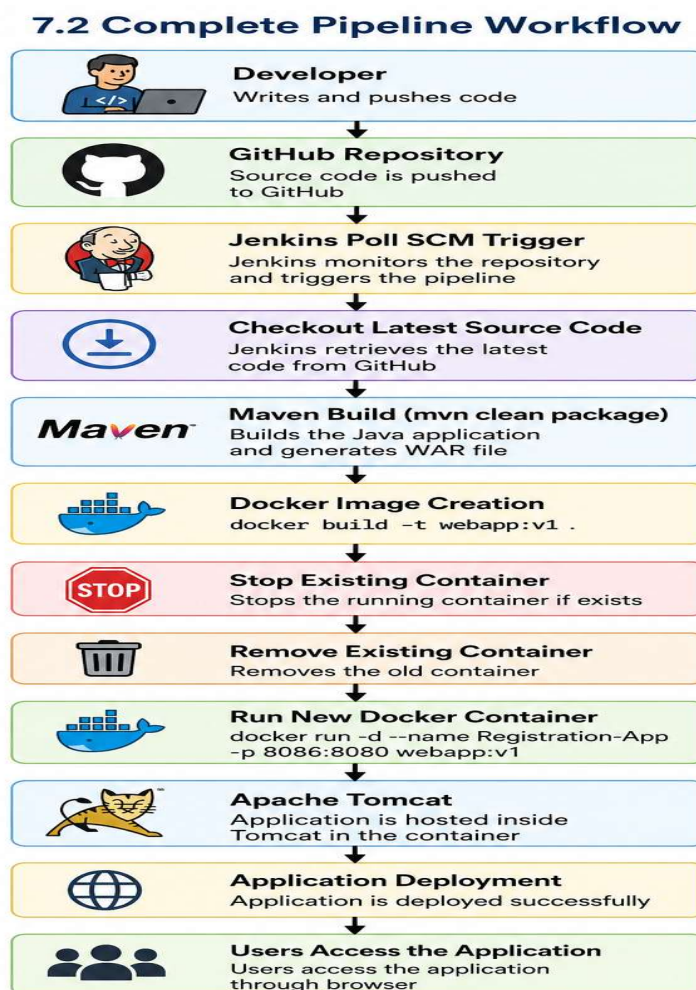
```
docker ps
```

The application is then accessed through a web browser.

<http://<EC2-Public-IP>:8086/Registration-App>

7.2 Complete Pipeline Workflow

The complete workflow of the project is summarized below:



Advantages of the Pipeline

- Automates the entire software build and deployment process.
- Reduces manual intervention and deployment errors.
- Ensures consistent application deployment using Docker containers.
- Enables rapid and reliable software delivery.
- Supports Continuous Integration and Continuous Deployment (CI/CD).
- Improves software quality and development productivity.
- Provides a scalable foundation for future cloud and Kubernetes deployments.

This pipeline workflow demonstrates the practical implementation of DevOps principles by integrating **GitHub**, **Jenkins**, **Docker**, and **Apache Tomcat** into a fully automated software delivery process. The automated pipeline ensures faster releases, improved reliability, and consistent deployments across different environments.

8.RESULTS

The implementation of the **Jenkins CI/CD Pipeline Using Docker** project was successfully completed, demonstrating an automated software build and deployment process using DevOps practices. The integration of **GitHub**, **Jenkins**, **Docker**, and **Apache Tomcat** enabled the application to be built, containerized, and deployed automatically with minimal manual intervention.

When the developer pushed the latest source code to the GitHub repository, Jenkins successfully detected the changes through **Poll SCM** and triggered the CI/CD pipeline. The pipeline automatically checked out the latest source code, built the Java web application using Maven, generated a Docker image using the Dockerfile, and deployed the application inside a Docker container. The deployment process completed successfully without any manual intervention.

The Docker container hosted the application on **Apache Tomcat**, making it accessible through a web browser using the configured server IP address and port number. Verification of the deployment confirmed that the application was running correctly and all pipeline stages executed successfully.

The project significantly reduced the time required for software deployment compared to the traditional manual approach. Automation eliminated repetitive tasks, minimized deployment errors,

and ensured a consistent deployment environment across different executions. Docker containerization also resolved dependency and environment-related issues by packaging the application and all required components into a single portable container.

Overall, the project successfully demonstrated the practical implementation of **Continuous Integration (CI)** and **Continuous Deployment (CD)** using Jenkins and Docker. The automated pipeline improved software delivery speed, enhanced deployment reliability, and simplified application management.

8.1 Achieved Results

- Successfully configured Jenkins for Continuous Integration and Continuous Deployment.
- Integrated GitHub with Jenkins to automatically trigger builds on source code changes.
- Successfully built the Java web application using Maven.
- Created Docker images automatically using a Dockerfile.
- Deployed the application inside a Docker container running Apache Tomcat.
- Verified successful deployment through a web browser.
- Reduced manual deployment effort through complete automation.
- Improved deployment consistency and reliability.
- Demonstrated real-world implementation of DevOps principles.

8.2 Project Outcome

The project achieved all the planned objectives by successfully implementing an automated CI/CD pipeline. The integration of GitHub, Jenkins, Docker, and Apache Tomcat enabled efficient software development, continuous integration, and automated deployment. The pipeline ensured faster releases, minimized human errors, and provided a reliable and scalable deployment environment. This project serves as a strong foundation for implementing advanced DevOps practices such as automated testing, cloud deployment, monitoring, and Kubernetes-based container orchestration in future enhancements.

Figure 13. Successfully Deployed Web Application

The screenshot shows a web browser window with the address bar displaying '3.80.99.45:8086/webapp/'. The page title is 'New user Register for DevOps Learning at Virtual TechBox Youtube Channel'. Below the title, a message says 'Please fill in this form to create an account.' followed by a registration form with five input fields: 'Enter Name' (placeholder: Enter Full Name), 'Enter mobile' (placeholder: Enter mobile number), 'Enter Email' (placeholder: Enter Email), 'Password' (placeholder: Enter Password), and 'Repeat Password' (placeholder: Repeat Password). Below the form, a text line states 'By creating an account you agree to our [Terms & Privacy](#).' followed by a 'Register' button. At the bottom of the form section, it says 'Already have an account? [Sign in](#).' Below the form section, the text 'Thank You, Happy Learning' and 'See You Again' is displayed.

New user Register for DevOps Learning at Virtual TechBox Youtube Channel

Please fill in this form to create an account.

Enter Name

Enter mobile

Enter Email

Password

Repeat Password

By creating an account you agree to our [Terms & Privacy](#).

Already have an account? [Sign in](#).

Thank You, Happy Learning

See You Again